Final Submission
Project Plan

R. Allenstein, A. Appling, J. Salazar, A. Walls - 05-05-2026 - v1

Table of Contents

1. Project Plan

Management plan

We have organized ourselves into different roles to split the project into manageable chunks of work. These roles include Data/ETL (Extract, Transform, Load), Catalog/Degree Requirements, Analytics/Report Generation, and UI/Integration/Setup. The data/ETL role will work on cleaning and organizing the data in CSV (Comma-Separated Values) format so that it is readable for the program and usable by the rest of the team. Catalog/ degree requirements will find and store the required courses for each major in the program and match them with the existing UO (University of Oregon) course catalog. Next, the analytics/ report generation will calculate the grade distribution throughout the courses, compare the percentage of passing grades for each instructor, and generate a report/ visualization of the data gathered. Lastly, the UI (User Interface)/ Integration/ Setup will connect all parts of the system to an I/O (input/ output) tool for the user to interact with.

Together, we have set up systems to ensure communication is kept up to date if anything arises. When assigning tasks, we chose within our skill set to distribute roles. For decision-making, we have all agreed on first proposing the decision and the reason for it, then stating whether we want to implement or deny the proposal. We hope to continue to do weekly meetings every Thursday and Sunday at noon to check in on progress or to answer any clarifications. These will be in person. Additionally, we have made a Google Doc to keep all meeting notes, which is shared with all members. Other forms of communication include: iMessage, Outlook, Gmail, Microsoft Word, and GitHub. GitHub will be used as our repository to keep the project in one location, and we can track our progress with each push of code.

2. Schedule Outline

1. **Finalize Project requirement understanding - Team - 4/8/26**
2. **Set up GitHub repo + environment - Joey - 4/8/26**
3. **Project Plan, SRS/SDS submission - Team - 4/15/26**
4. **Parse grade history - Reid - 4/19/26**
5. **Parse degree plan - Joey - 4/19/26**
6. **Implement reconciliation rules (logic to sort data) - Joey - 4/18/26**
7. **Data cleaning and normalization - Reid, maybe Adriana or Joey -**
8. **Data aggregation and analytics engine - Adriana - 1/1/26**
9. **Visualization layer (graphs) - Abby - 5/2/26**
10. **Report generation - Adriana and Abby - 5/1/26**
11. **Web interface - Abby - 5/2/26**
12. **End-to-end integration - Abby, maybe team - 5/3/26**
13. **System testing and validation - Team - 4/8-5/3/26**
14. **Documentation - Team - 4/8-5/4/26**
15. **Final report - Team - 5/4/26**
16. **Final presentation - Team - 5/5/26**
17. **Submission - Team - 5/4/26**

3. Monitoring and Reporting

As a group, we decided that we would do weekly check-ins to make sure we are on track. We also have the milestones, which we will use to track the progress of the overall project. Additionally, on the GitHub repository, there is a display of the history for every change made to the project; thus, we can use it to keep track of who is actively contributing to the project in real time.

3.1 Meeting Record

The team met regularly on Thursdays and Sundays from about 12:00 PM to 1:00–1:30 PM. Meetings were used to divide work, check progress, discuss blockers, and decide what needed to be completed before the next milestone.

April 12: Chose Computer Science, Psychology, and Architecture as supported majors, agreed on the degree CSV format, and split initial planning/SDS work.

April 16: Began implementation work across assigned project areas.

April 17–18: Worked on degree requirements, course matching, and checking required courses against available grade data.

April 19: Discussed data format and shifted work toward other project needs as degree/data matching progressed.

May 3: Focused on frontend/backend integration, admin upload testing, README updates, SDS updates, and presentation preparation.

4. Build Plan

We decided to split the project into four main parts so it stays manageable for each team member. Each part handles one major job in the system, which makes it easier for our team to start building right away.

First, we needed to understand the two main things our project depends on.

- **Grade Data: Figure out how we're organizing it, and decide which information matters most in the CSV**
- **Degree Requirements: Confirm the required classes for Computer Science, Psychology, and Architecture.**

Next, we will get those two parts ready to work together.

- **Degree Files: We will create a CSV file for each major using the required format: Year, Term, Numb, Subj, Title**
- **Course matching: We will make sure the classes in those files line up with how courses appear in the grade dataset.**

Then, we can implement the system's main logic.

- **System Logic: The Program will take a major and a year range as input, find the required courses for that major, and pull the matching grade data.**
- **Grade Analysis: It will calculate grade distributions for the covered courses.**
- **Instructor Comparison: It will compare instructor grade data for those courses so users can see which instructors tend to give higher grades.**

Once that is working, we will connect everything into one full system.

- **Integration: The interface, analytics, and data parts will be combined.**
- **User workflow: The user will be able to make a selection and get a report back.**

At the end and likely throughout, we will test everything.

5. Rationale

We chose this plan so everyone could have an idea and start working early, while still keeping the project organized. Even though the system comes together in stages, each team member has a clear starting task from the beginning.

We also chose this order because some parts need to be checked and connected before the full system will work correctly. For example, the grade data and degree requirements both need to be checked early, and the course matching needs to be checked so we know the rest of the project is using the right classes.

The biggest risk is making sure the degree requirements are correct. If we leave out a required class or include the wrong one, the system could give a student a misleading picture of that major. In a real situation, missing requirements could cause problems for graduation planning, so we want to be careful about that from the start.

Another risk is course matching between the catalog and the grade dataset. Even if the degree requirements are correct, the system still will not work well if the class names or numbers don't line up with the dataset. To reduce that risk, we are checking course matching early so we know the system is pulling data for the right courses.

There is also some risk when combining everyone's work, since different people are building different parts of the project. To help with that, we plan to stay organized and use an Agile-style chart to track what each person is working on and avoid merge conflicts.

Max GPA
Software Design Specification

Table of Contents

1. SDS Revision History

Date	Author	Description
04/09	Reid Allenstein	Initial document creation
04/13	Joey Salazar	Titling
04/15	Reid Allenstein	Page Numbering and Design Rationale
04/15	Abigail Appling	Overall design description
04/15	Reid Allenstein	Major Subsystems
04/15	Adriana Walls	Table of Contents
04/15	Salazar and Allenstein	Software Rationale for Software Modules
04/15	Adriana Walls	Roles and Interface Specifications for Software Modules
04/15	Abigail Appling	Software Architecture for Software Architecture
04/15	Joey Salazar	Dynamic Models of Operational Scenarios (Use Cases)
04/15	Reid Allenstein	References and Acknowledgements

04/15	**Abigail Appling**	**Design Rationale for Software Architecture**
04/15	**Adriana Walls**	**Functionality and Interactions for Software Architecture**
04/15	**Adriana Walls**	**SDS Revision History**
04/15	**Joey Salazar**	**UI/Integration models and rationale for Software Modules**
04/15	**Adriana Walls**	**Roles and Interface Specification for Software Modules**
05/04	**Reid Allenstein**	**Updated Dynamic Models to UML Sequence diagrams**
05/04	**Abigail Appling**	**Updated Design Rationale**

2. System Overview

MaxGPA will be built around four major subsystems that each handle one distinct job. Data/ETL is responsible for reading in the raw grade history CSV files provided by the university, cleaning the data, and normalizing it so the rest of the system can use it reliably. Catalog/Degree Requirements handles finding and storing the required courses for each of the three majors, Computer Science, Psychology, and Architecture, and matching those courses against the UO course catalog. Analytics/Report Generation takes the cleaned, matched data and calculates grade distributions across all required courses, compares instructor performance, and produces the visualizations and reports the student sees. Finally, UI/Integration/Setup connects all three of those parts into a single working system that a student or administrator can actually interact with.

3. Software Architecture

1. The Components of this project are

1. **Grade Record Store**
    a. **holds all cleaned and normalized grade data.**
2. **Degree Plan Store**
    a. **holds the structured course requirements for each supported major.**
3. **Grade History Loader**
    a. **handles reading the grade history file.**
4. **Degree Plan Loader**
    a. **Handles reading degree plan files formatted from the UO Catalog.**
5. **Grade Distribution Engine**
    a. **retrieves required course lists from the degree plan store and corresponding grade records.**
6. **Student Report Interface**
    a. **presents the student-facing side of the system.**

2. Functionality:

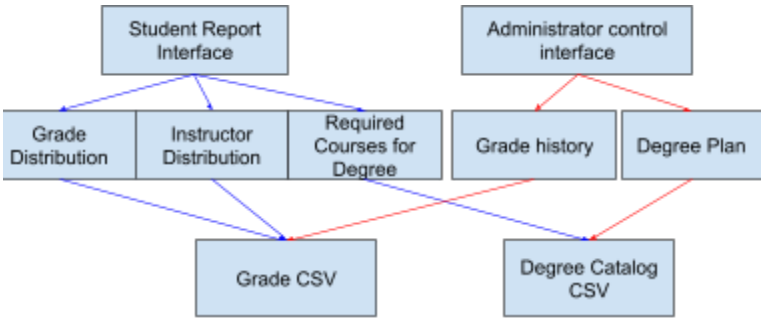

Figure 1: Architectural diagram where the raw CSV data is being parsed, normalized, analyzed, compared, and displayed to the user interfaces.

The MaxGPA system is composed of four primary components, each responsible for a distinct aspect of data processing, requirement management, analysis, and report generation.

1. **Grade Data Manager**
    a. **Responsible for ingesting, validating, and transforming raw grade data into a structured format suitable for analysis. It ensures data consistency by cleaning and normalizing course and instructor information and provides access to processed grade records for other components.**
2. **Degree Requirement Manager**
    a. **Maintains structured representations of degree requirements for each supported major. It organizes course requirements into required and elective categories and provides access to these course sets for analytical use.**
3. **Grade Analytics Subsystem**
    a. **Serves as the core computational component of the system. It combines grade data and degree requirements to produce analytical outputs such as grade distributions and instructor comparisons. It's responsible for generating structured results based on user-defined parameters.**

3. Interactions:

The system components interact in a structured and modular manner to achieve the overall functionality of generating grade-based academic insights. The grade analytics subsystem, acting as the central component, interacts with both the grade data processor and degree requirement manager to retrieve the necessary data for analysis. The grade data processor provides cleaned and structured grade data, while the degree requirement manager supplies the relevant course requirements for a selected major.

These interactions are designed so that each component depends only on well-defined services provided by others, ensuring that data processing, requirement management, and analysis remain loosely coupled and independently maintainable.

4. Design Rationale:

- **Separate data sources stay separate.**
  - **The grade history from UO and the course requirements from the UO catalog are in different formats, origins, and timelines.**
- **Each component has exactly one job.**
  - **The loaders only validate and write. The stores only hold data. The Grade Distribution engine only analyzes. The student report interface and the administrator control interface only present information and collect user input. This allows each team member to work independently without waiting for others.**
- **No interface touches raw data directly**
  - **The student report interface only talks to the grade distribution engine.**
  - **The administrator control interface only talks to the loaders. This means data validation and cleaning always happen in one controlled place, and a change to how data is stored never requires a change to the interface code.**
  - **The UI may start workflows that use grade data, but it does not parse raw grade rows or calculate grade distributions itself. The UI collects user input, sends requests to the analysis/data logic, and displays processed results.**
- **Alternative Designs considered:**
  - **Terminal-only interface.**
    - **We initially considered keeping the system entirely terminal-based since the analysis engine runs in the terminal. We rejected this because a terminal interface makes it much harder for non-technical students and administrators to use the system, and displaying bar charts in a terminal is also not practical.**
  - **Flask vs. raw Python HTTP server**
    - **We considered using Python's built-in http.server module to avoid any external dependencies. We rejected this because implementing routing, templates, file uploads, and session management would require significantly more complex code. Flask was a better option as it was also listed in the project specification.**

4. Software Modules

4.1. Grade Data Subsystem

A. Role:

The grade data processor is responsible for ingesting, validating, and transforming raw grade data into a structured format that other system components can efficiently query. Beginning with parsing CSV input files containing grade records, it then validates each entry to ensure completeness and correctness. Invalid or malformed records are rejected, while valid data is normalized to resolve inconsistencies in course identifiers and instructor names. After normalization, the processed records are stored in a structured database. The module also exposes query functionality, allowing other components to retrieve grade data filtered by course, instructor, or year.

B. Interface Specification:

It abstracts the underlying data storage and processing logic, exposing only query-based operations to other modules.

- **Load grade data**
  - **Load and prepare grade data from an external source**
  - **Returns success/failure status**
- **Check for courses with data**
  - **If a course has only asterisks (*), then data has not been provided; thus, ignore**
  - **Otherwise, continue with parsing**
- **Get the grade by course**
  - **Return grade data for a specified course**
- **Make course ID**
  - **Return the course ID subject and concatenate with the course ID number**
- **Normalize grade row**
  - **From the grade or degree plan CSV, check that headers match and call make course ID**
  - **Return the course ID and the catalog title of the course**
- **Get grades by instructor**
  - **Returns grade data filtered by instructor**
- **Get grades by year**
  - **Returns grade data filtered by course and time range**

C. Static Model:

The entry point that scrapes or reads raw grade records. They are then normalized into and persist in the grade database.

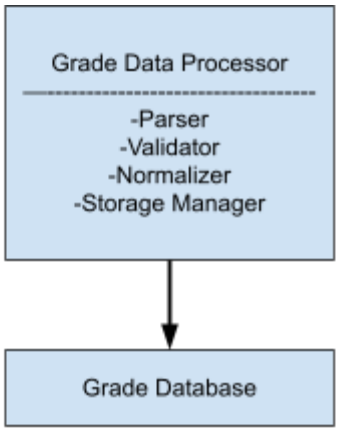

Figure 2: Static model for Grade Data Processor

D. Dynamic Model:

At runtime, the collector triggers the parser, which validates each row and either discards it or hands a record to the database. The query interface handles on-demand lookups from other modules.

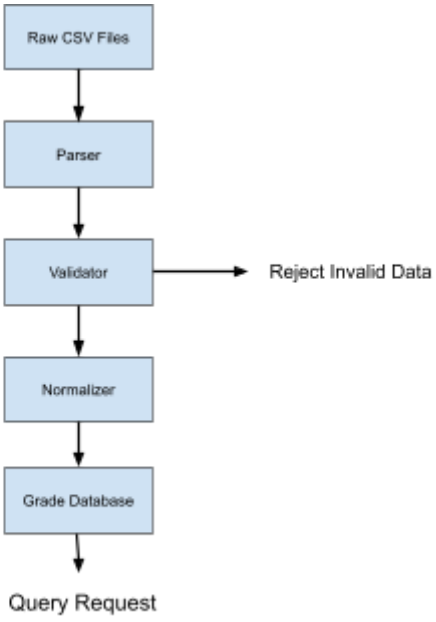

Figure 3: Dynamic model for grade data process

E. Rationale:

The Grade Data module is designed around the principle of data normalization and data hiding. The decision to separate the system into data collection, parsing, storage, and query interface makes it so that raw grade data is cleaned and standardized before it is accessed by other parts of the system. This also isolates the file format, so any input changes don't affect modules that rely on the data.

An alternative design was to be a singular structure where raw grade data is read and processed directly by the analysis module without a separate parsing and storage layer. This idea was rejected because it would tightly couple file formats with analytics logic, making the system harder to maintain and more error-prone when the input formats change.

4.2. Degree Requirements Subsystem

A. Role:

The degree requirement manager maintains and organizes degree requirements for each major, providing structured access to required and elective courses. It will parse CSV files containing course requirements and will categorize each course as either required or elective. The module then constructs structured degree plans and stores them in a database indexed by major. It provides query capabilities that allow the components to retrieve the full set of required and elective courses for a given major, enabling integration with the analytics process.

B. Interface Specification:

This abstracts the parsing and organization of requirement data.

- **Load degree requirements**
  - **Loads and initializes degree requirement data**
- **Get the required courses**
  - **Returns a list of required courses for a given major**
- **Get elective courses**
  - **Returns a list of elective courses**
- **Get the full degree plan**
  - **Returns the complete degree structure for a major**
- **Find course match**
  - **Match grade CSV data to required degree plan CSV data**

C. Static Model:

Takes extracted raw course listings from the UO catalog. Then interprets those listings into structured course requirements, which are organized by major, which lets the Grade Analysis subsystem ask which courses fulfill a given major.

D. Dynamic Model:

Takes the major's degree catalog as a csv, then parses it into the correct format. The query interface returns the required and elective course lists for a given major on command.

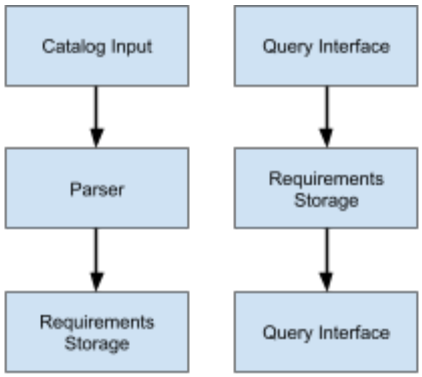

Figure 4: Dynamic model of storing catalog data and accessing catalog data

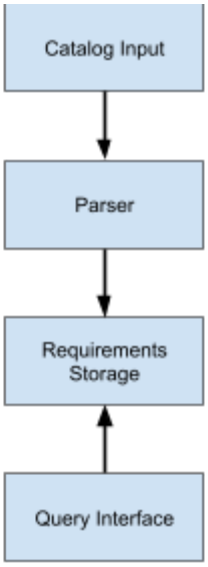

Figure 5: Static model for degree requirement data process

E. Rationale:

The Degree Requirements module is designed to keep track of the required courses for each major separately from the grade data. This makes it easier to check that the system is using the correct classes before any analysis happens. Keeping this module separate also helps prevent mistakes when linking required classes to the grade dataset and makes it easier to update degree requirements later without changing the rest of the system.

The alternative approach was to integrate degree requirements directly into the grade data module. The idea was rejected because it would mix static curriculum data with dynamic

grade data, reducing modularity and making updates to degree requirements more difficult and error-prone.

4.3. Grade Analysis Subsystem

A. Role:
The grade analytics subsystem serves as the core processing unit of the system, combining grade data and degree requirements to generate analytical insights and reports. It accepts user-defined parameters such as major and year range, then retrieves the relevant course requirements from the degree requirement manager. Using this course set, it queries the grade data processor to obtain corresponding grade data, which is then filtered based on the specified time range. The engine performs computations to generate grade distributions and instructor comparisons, and compiles the results into a structured format suitable for reporting. These results are then passed to the interface layer for presentation.

B. Interface Specification:
This acts as the central computation module of the system.

- **Generate analytics**
  - **Accepts selected major, year range, degree plan data, and grade data**
  - **Returns a structured analysis result**
- **Get grade distribution**
  - **Computes grade distributions for a set of courses**
- **Instructor distribution**
  - **Enables instructor comparison by connecting the professor to the grades and the course**
- **Rank instructors**
  - **Those with a higher percentage of A or pass will be ranked as the best option for students to consider**
- **Summarize results**
  - **Converts analysis outputs into a structured summary**

C. Static Model:

The module takes a student's major and declared courses as input. pulls the requirement list from the Degree Requirements subsystem. Retrieves average grades per course-professor pair from the Grade Data subsystem. scores and sorts the valid combinations, and shows the final ranked output.

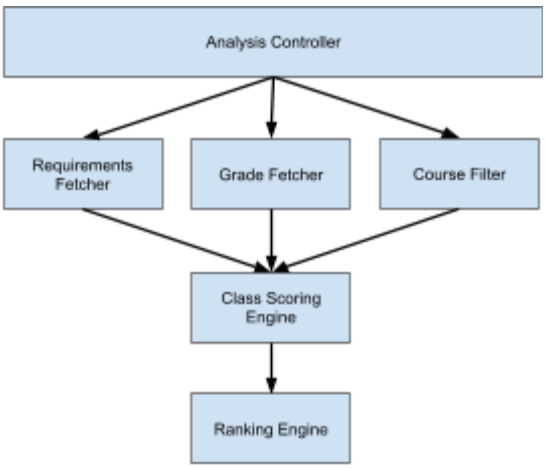

Figure 6: Static Model of Grade Analysis process

D. Dynamic Model:

The module takes the selected major and year range, retrieves the major requirements, fetches matching grade records, calculates grade distributions, compares instructors, and returns structured report data for the UI.

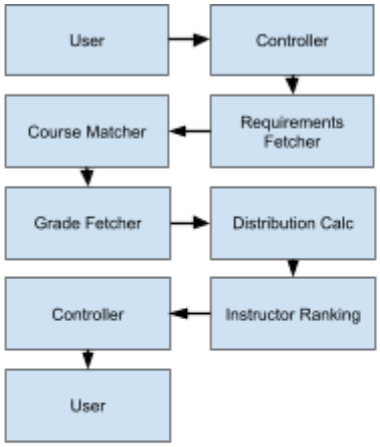

Figure 7: Dynamic model of Grade Analysis process

E. Rationale:

The Grade Analysis module is designed around filtering and ranking the data from the grades and degree requirements. It takes user input and returns recommendations based

on their needs. Each stage of the analysis narrows the data down until it is left with the final ranking requirements. While doing this, the module maintains a separation between the data retrieval and decision logic so that the analysis focuses entirely on decision logic.

An alternative design considered was combining data retrieval and analysis logic into a single process. This approach was rejected because it would blur responsibilities within the module, making the system harder to test, debug, and extend, particularly if new analysis strategies are introduced.

4.4. UI/Integration

A. Role:

The UI module manages user interaction, collects input parameters, and presents processed analytical results in a readable format. It captures user input, including selected major and year range, and validates the input to ensure it meets expected formats and constraints. Once validated, the module forwards the request to the grade analytics subsystem and waits for the processed results. Once received, it formats the data into a clear and readable structure, such as tables and/or summaries, and presents the final output to the user.

B. Interface Specification:

Provides services for initiating analysis requests and delivering formatted results. It serves as the entry and exit point for system interaction.

- **Submit request**
  - **Accepts user parameters and initiates analysis**
- **Validate input**
  - **Ensures input meets required constraints**
- **Get analysis results**
  - **Retrieves processed results from the data analytics subsystem**
- **Format report**
  - **Converts raw results into display ready format**

C. Static Model:

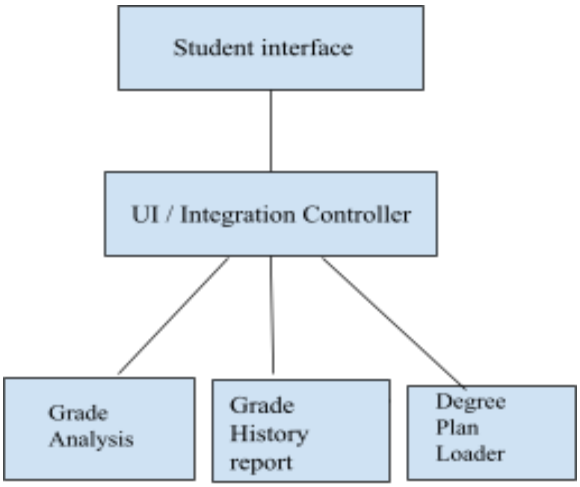

Figure 8: Static model of UI

D. Dynamic Model:

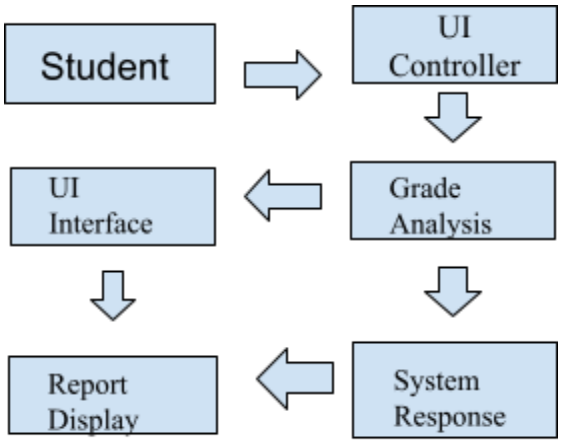

Figure 9: Dynamic model of the UI

E: Rationale:

The UI / Integration module is responsible for turning the system's internal results into a usable workflow for the student. Our project depends on multiple parts working together: selecting a major, choosing a year range, pulling the matched data, and showing the final report. Because of that, this module is designed to coordinate the flow between subsystems and present the results clearly to the user. This design makes the system easier to use and helps ensure that changes to the interface do not affect how the data is cleaned, matched, or analyzed.

An alternative we considered was storing the uploaded grade data in a database instead of reading from the CSV files during analysis. This could have worked because the app could query courses, instructors, and year ranges more quickly after the data was loaded. This would have made the system more scalable, but we decided not to use this approach because it would require us to design the database tables and add extra steps for the admin. Using CSV files kept the system simpler to install, test, and explain.

5. Dynamic Models of Operational Scenarios (Use Cases)

Every major Use Case should be described with a dynamic model such as a UML sequence diagram.

Use cases:

Use case 1. Generating grade analysis for a user's major
This use case occurs when a user requests an analysis of grade data for a given major in order to maximize their GPA. The system retrieves the degree requirements and the matching grade data before ranking them for the user.

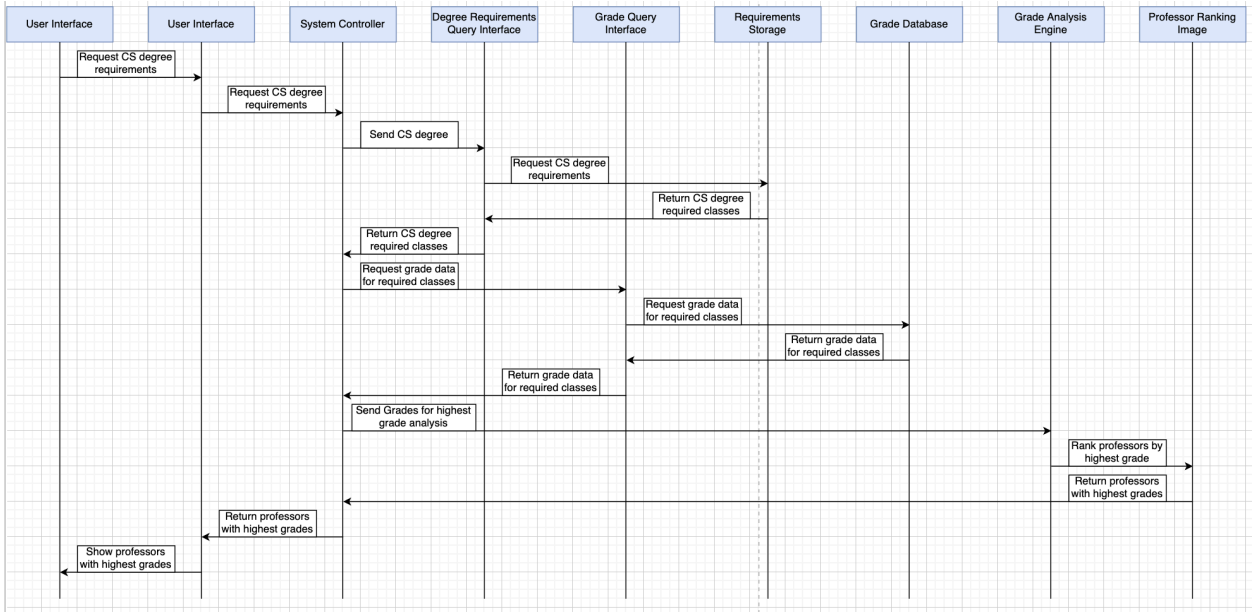

Figure 10: Model for generating grade analysis

Use case 2. Retrieving degree requirements for a major
This use case occurs when the system retrieves the degree requirements for a given major.

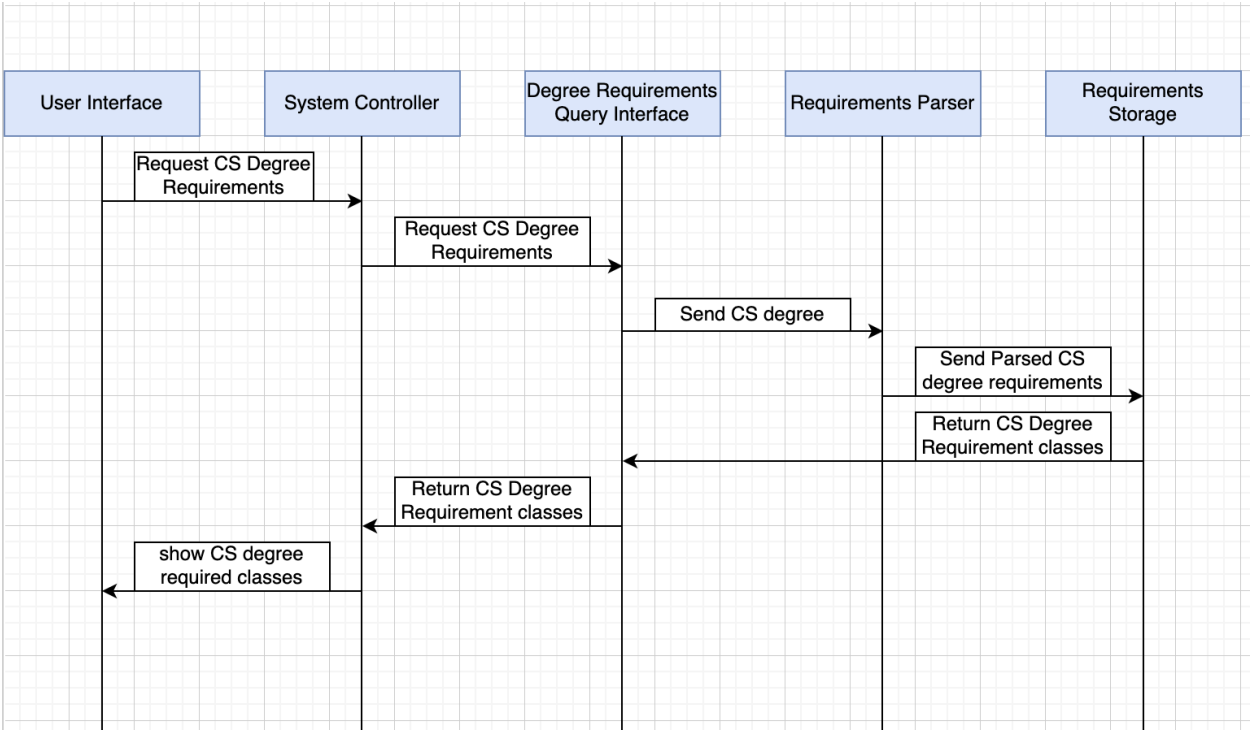

Figure 11: Model for retrieving major specific requirements

Use case 3. Loading and querying data

This use case occurs when the administrator uploads the cleaned grade history CSV and the system saves it for later analysis.

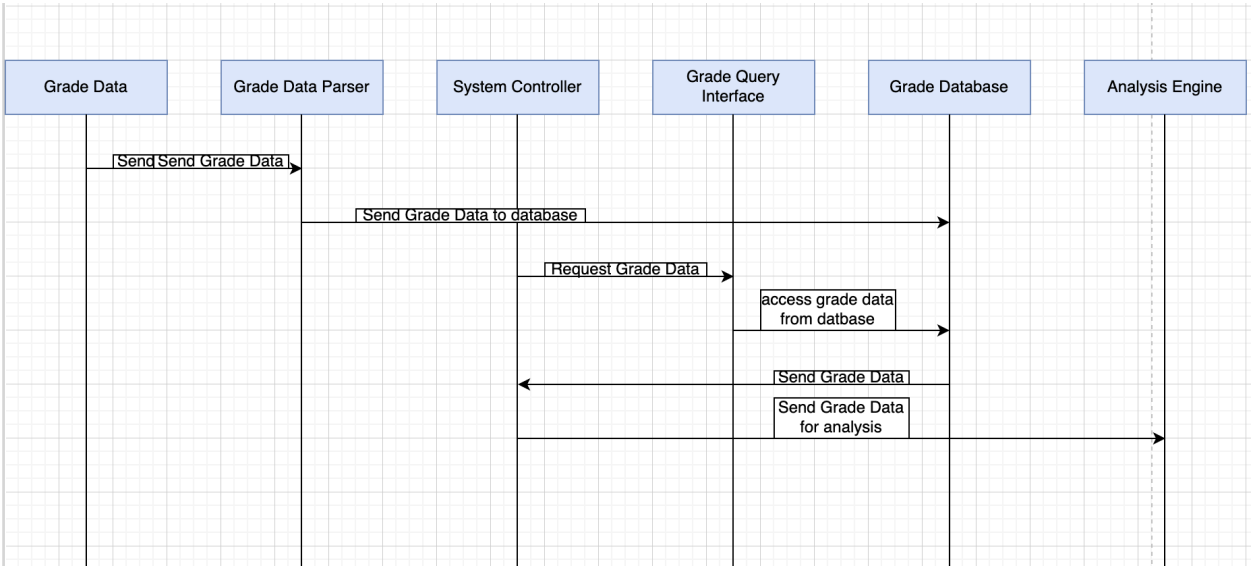

Figure 12: Sequence Diagram for loading and querying grade data

6. References

1. **University of Oregon Catalog. Computer Science Major Requirements.** https://catalog.uoregon.edu/arts-sciences/school-computer-data-sciences/computer-science/ug-computer-science/#degreeplantext
2. **University of Oregon Catalog. Psychology Major Requirements.** https://catalog.uoregon.edu/arts-sciences/natural-sciences/psychology/ug-psychology/#degreeplantext
3. **University of Oregon Catalog. Architecture Major Requirements.** https://catalog.uoregon.edu/design/arch-env/architecture/barch-arch/#requirementstext
4. **University of Oregon historical grade data files.**
5. **CS 422 Project 1 / Initial Submission handout and evaluation materials.** https://classes.cs.uoregon.edu/26S/cs422/Project_Evaluation.html
6. **CS 422 SDS Template.** https://classes.cs.uoregon.edu/26S/cs422/Handouts/Template-SDS.pdf

7. Acknowledgements

We acknowledge the University of Oregon course catalog and university grade-history data as the primary data sources used in this project. We also acknowledge the CS 422 course materials, including the SDS template and project evaluation handout, for guiding the structure of this document. Team members collaborated on the planning, design, and documentation of this project.